Role Playing Variables: Immutable Objects I

- Two volunteers to play the role of variables
- Two operations in our world:
  - = is the "grab" operator
  - **print** is the "shout out what you're holding operator"
- Three *immutable* objects in our world:
  - two green balls, one red ball
- Let's run this code:

```
person_a = "green ball"
person_b = "green ball"
print(person_a)
print(person_b)
person_a = "red ball"
print(person_a)
print(person_b)
```

- What was the output of this program?

Variables Can't Do This



http://www.pups-seeking-homes.com/blog/home/category/Fun+Stuff!

Role Playing Variables: Immutable Objects II

- Let's alter the world slightly and run the same code again:
- Two operations in our world:
  - = is the "grab" operator
  - **print** is the "shout out what you're holding operator"
- Two *immutable* objects in our world:
  - one green ball, one red ball
- Code:

```
person_a = "green ball"
person_b = "green ball"
print(person_a)
print(person_b)
person_a = "red ball"
print(person_a)
print(person_b)
```

- Was the output of the program different?

Role Playing Variables: Immutable Objects III

- Now let's keep the same world and alter the code slightly:
- Two operations in our world:
  - = is the "grab" operator
  - **print** is the "shout out what you're holding operator"
- Two *immutable* objects in our world:
  - one green ball, one red ball
- Code:

```
person_a = "green ball"
person_b = person_a
print(person_a)
print(person_b)
person_a = "red ball"
print(person_a)
print(person_b)
```

- Was the output of the program different?

Role Playing Variables: Immutable Objects

- In the physical model we've defined, does it matter if we use the same green ball or two different green balls?
- *Why* does it not matter?
  - We have no way to distinguish between the two green balls
  - Since we have no way to alter the green ball we never will need to distinguish between them anyway
  - `Python` feels this way about strings and ints and floats

Role Playing Variables IV: Immutable Objects

How about passing an immutable argument to a function?

```
def change_param(some_person):
    print(some_person)
    some_person = "blue ball"
    print(some_person)
    return None

# ======================
def main():
    person_a = "green ball"
    print(person_a)
    person_a = "red ball"
    print(person_a)
    change_param(person_a)
    print(person_a)
```

- Is there any way the function can alter the value in main's `person_a` variable?

Role Playing Variables

- Is there any way we can make `person_a` be holding a different object *without* assigning something to `person_a`?
  - No, if you want `person_a` to be holding a different object you have to use the assignment operator on `person_a`:

    ```
    person_a = something_else
    ```

- Is there any way we can alter the object `person_a` is holding if it is immutable?
  - No, immutable means unchangeable.
  - If you try to change it the immutability police will crash your program.

Role Playing Variables: Mutable Objects I

- Six objects in our world (bins are mutable):
  - bin, bin, green ball, green ball, red ball, blue ball
- Three operations in our world:
  - = is the "grab" operator
  - **print** is the "shout out what you're holding operator"
  - `append` allows us to put things into bins
- Code:

```
person_a = []
person_b = []
print(person_a)
print(person_b)
person_a = "red ball"
print(person_a)
print(person_b)
```

- Assignment still does what it always does!

Role Playing Variables: Mutable Objects II

- Six objects in our world (bins are mutable):
  - bin, bin, green ball, green ball, red ball, blue ball
- Three operations in our world:
  - = is the "grab" operator
  - **print** is the "shout out what you're holding operator"
  - `append` allows us to put things into bins
- Code:

```
person_a = []
person_b = []
print(person_a)
print(person_b)
person_a.append("red ball")
print(person_a)
print(person_b)
```

Role Playing Variables: Mutable Objects III

- Same world:

```
person_a = []
person_b = person_a
print(person_a)
print(person_b)
person_a.append("red ball")
print(person_a)
print(person_b)
person_a = "blue ball"
print(person_a)
print(person_b)
```

Role Playing Variables: Mutable Objects

- So, clearly it *does* matter if we use the same bin or different bins in our physical representation
  - Since we can modify a bin (by inserting/removing things from it) each bin can have different contents
  - We need to keep track of the current contents of each of the different bins

Role Playing Variables: Mutable Objects IV

- Code:

```
def append_blue_ball(some_bin):
    some_bin.append("blue ball")
    print(some_bin)
    return None

# ======================

def main():
    person_a = []
    print(person_a)
    person_a.append("red ball")
    print(person_a)
    append_blue_ball(person_a)
    print(person_a)
```

- Note that the function is not assigning to `some_bin`, it is calling a method on the object that variable refers to.

Role Playing Variables V: Mutable Objects

- Same world:

```
person_a = []
person_b = person_a
person_a.append("red ball")
person_a.append("blue ball")
print(person_a)
print(person_b)
person_a[0] = "green ball"
print(person_a)
print(person_b)
```

- The indexing operator allows us to access and alter the contents of the *object* referred to by `person_a`
- `person_b` is still holding the *same* object, the object has just been altered slightly.

Mutable vs. Immutable

- An assignment statement changes the object referred to by the variable on the left to the value of the expression on the right

  ```
  variable = <expression>
  ```

  - i.e. it simply points the variable at a new object, it does not do anything to the old object

- When you call a method on a *mutable* object, the method *can* alter the object

  ```
  some_object.some_method()
  ```

- When you pass a *mutable* object to a function, the function *can* alter the object

- You *can* use the index operator to assign values to different locations in a list (i.e. to mutate the list)

  ```
  some_list[i] = new_value
  ```